

Rockchip_RK3588_HDMI_To_CSI_Dual_Mipi_Developer_Guide

文件标识: RK-KF-YF-755

发布版本: V1.0.1

日期: 2023-12-28

文件密级: ☐绝密 ☐秘密 ☐内部资料 ☒公开

免责声明

本文档按“现状”提供, 瑞芯微电子股份有限公司 (“本公司”, 下同) 不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因, 本文档将可能在未经任何通知的情况下, 不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标, 归本公司所有。

本文档可能提及的其他所有注册商标或商标, 由其各自拥有者所有。

版权所有 © 2023 瑞芯微电子股份有限公司

超越合理使用范畴, 非经本公司书面许可, 任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部, 并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址: 福建省福州市铜盘路软件园A区18号

网址: www.rock-chips.com

客户服务电话: +86-4007-700-590

客户服务传真: +86-591-83951833

客户服务邮箱: fae@rock-chips.com

前言

本文档是基于RK3588 Android平台，开发HDMI转MIPI-CSI2双MIPI合成模式的帮助文档。

概述

本文档主要提供RK3588平台，HDMI/DP转MIPI-CSI应用场景，使用双MIPI PORT来传输高分辨率图像（4K60）的配置指导。

产品版本

芯片名称	内核版本
RK3588	Linux 5.10

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	范建威	2023-4-3	初始版本
V1.0.1	范建威	2023-12-28	支持IC增加RK628F

目录

Rockchip_RK3588_HDMI_To_CSI_Dual_Mipi_Developer_Guide

- 1. RK3588双MIPI方案概述
 - 1.1 问题描述
 - 1.2 流程框图
 - 1.3 支持双MIPI转接芯片描述
 - 1.4 RK3588 MIPI RX规格
- 2. 关键配置
 - 2.1 内核补丁
 - 2.2 驱动修改
 - 2.3 DTS配置
 - 2.4 转接芯片配置说明
- 3. 调试命令
 - 3.1 查看拓扑
 - 3.2 抓图命令
- 4. 常见问题
 - 4.1 dts配置是否需要配置两个链路？
 - 4.2 应用层是否需要重新适配？
 - 4.3 抓图异常
 - 4.3.1 抓不到图且串口没有报错
 - 4.3.2 抓不到数据MIPI相关报错

1. RK3588双MIPI方案概述

1.1 问题描述

RK3588平台支持HDMI转MIPI-CSI做HDMIIN功能，但受限于转接芯片的DPHY的规格，无法实现一个MIPI-DPHY接口实现接收4K60 的HDMI-IN图像。

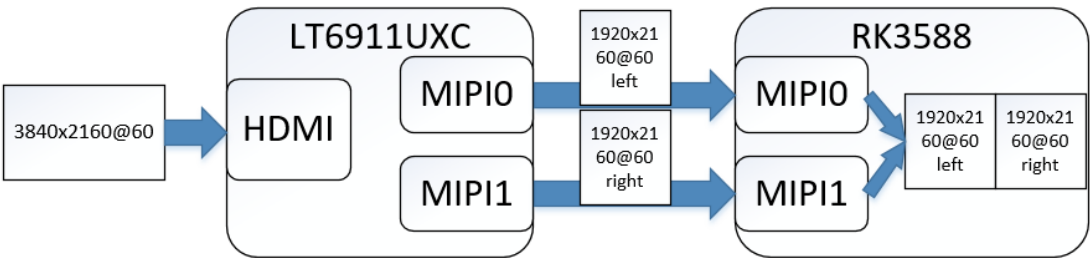
以MIPI传输YUV422格式图像为例，需要接近10Gbps的带宽，按4lane传输，每条lane则需要2.5Gbps的吞吐量，RK3588的DPHY接口均满足该要求，但是市面很多转接芯片不满足该规格，如：LT6911UXC/C，LT7911D等，但这类转接芯片内部有两个MIPI PORT，可以实现将4K60的图像分为两个MIPI PORT共计8lane传输。

本文档主要描述在外部MIPI TX按照两个MIPI PORT传输左右两半图像，RK3588主控端如何使用两个MIPI口接收拼接的方案配置指导。

注：RK平台该方案仅支持两个4lane的MIPI PORT分别传输左右两半的图像，不支持8lane的模式，即不支持按Byte或是Pixel分别在两个MIPI PORT之间交替传输的方式。

1.2 流程图

总体流程如下所示，4K60的HDMI图像给到LT6911UXC等转接芯片接收，转接芯片将3840x2160的图像拆分成1920x2160左右两半图像，分别通过两个4lane的MIPI PORT传输，主控由两个MIPI分别接收左右两半的1920x2160@60的图像，由主控完成拼接，存储4K60图像。



1.3 支持双MIPI转接芯片描述

目前龙讯部分的转接芯片和RK628F支持双MIPI的模式传输，规格如下：

转接芯片	功能	PHY 吞吐率	双MIPI模式
RK628F	HDMI to CSI HDMI to DSI	DPHY:1.5Gbps/lane	4K60YUV422
LT6911UXC	HDMI to CSI	DPHY: 2Gbps/lane	4K60YUV422 4K60RGB888
LT6911UXE	HDMI to CSI	DPHY:2.5Gbps/lane	4K60RGB888 5K60YUV422
LT7911D	DP/Type-c to CSI	DPHY:1.5Gbps/lane	4K60YUV422
LT7911UXC	DP/Type-c to CSI	DPHY:2.5Gbps/lane CPHY:2.5Gbps/trio(5.7Gbps)	4K60RGB888 5K60YUV422

1.4 RK3588 MIPI RX规格

RK3588主控支持双MIPI接收高分辨率图像进行拼接，RK3588 MIPI RX规格如下

芯片型号	MIPI RX数量	最大分辨率(PHY规格)	支持格式
RK3588s	3x4lane	DPHY: 1xDPHY-v1.2: 2.5Gbps/lane x 4lane DCPHY: (1)DPHY 2xDPHY-v2.0: 2.5Gbps/lane x 4lane (2)CPHY 2xCPHY-v1.1: 5.7Gbps/lane x 3lane	YUV422/RGB888/ RGB666/RGB565
RK3588	4x4lane	DPHY: 2xDPHY-v1.2: 2.5Gbps/lane x 4lane DCPHY: (1)DPHY 2xDPHY-v2.0: 2.5Gbps/lane x 4lane (2)CPHY 2xCPHY-v1.1: 5.7Gbps/lane x 3lane	YUV422/RGB888/ RGB666/RGB565

注：CPHY描述带宽吞吐率单位为sps，与bps的换算：1 Gsps = 2.28 Gbps。

2. 关键配置

2.1 内核补丁

vicap驱动需要添加支持双MIPI接收的补丁，RK3588 Android12 R12SDK版本可以使用如下补丁包：

[rk3588_lt6911uxc_dual_mipi_base_R12.rar](#)

内核vicap主要包括如下提交：

```

media: rockchip: vicap support combine two mipi to one dev

Signed-off-by: Zefa Chen <zefa.chen@rock-chips.com>
Change-Id: Iba3e83d0bc1433458d56c2542c3224ffee127b90

phy: rockchip: csi2-dphy: logic node of mipi phy can control all hw of mipi phy

Signed-off-by: Zefa Chen <zefa.chen@rock-chips.com>
Change-Id: I30cc62cc1d28c4219e9e5c5ccd77fa9f589e63af

include: rk-camera-module: support get capture info

Signed-off-by: Zefa Chen <zefa.chen@rock-chips.com>
Change-Id: Ic1f117afcb53b035086f6835deb0ccf2733ee972

include: rkCIF-config: support set multi csi info

Signed-off-by: Zefa Chen <zefa.chen@rock-chips.com>
Change-Id: I8509ed952b9554659c0238024a383e547620825b

ARCH: arm64: rk3588 separate the node of csi2 and hw, logical and physical nodes
are separated

Signed-off-by: Zefa Chen <zefa.chen@rock-chips.com>
Change-Id: Ibb75cc466452aedff8f50d29331b191d2fbd922a

ARCH: arm64: rk3588 remove csi2_dcphy0/csi2_dcphy1

1. all logic node of mipi phy use csi2_dphy0,1,2,3...
2. all logic node of mipi phy can get all hw of mipi phy
3. the links between logic and hw is determined by upper level equipmen

Signed-off-by: Zefa Chen <zefa.chen@rock-chips.com>
Change-Id: Icc0cb88c3294a119431ac24b0043e44e34b1b292

```

2.2 驱动修改

转接芯片驱动需要添加双MIPI模式的支持，可以参考LT6911UXC的修改，需要在转接芯片的驱动识别双MIPI的模式和单MIPI的模式，并配置相关信息，VICAP驱动通过IOCTL获取相应的配置信息。

LT6911UXC通过寄存器的lane数来识别是否是双MIPI模式，只有4K60的分辨率才会切换到双MIPI的模式，4K30及以下的分辨率是按照单MIPI的方式传输。LT6911相关的修改可参考如下，读取dts相关的信息，配置双MIPI模式对应的结构体rkmodule_multi_dev_info，VICAP驱动通过IOCTL获取该结构体，实现双MIPI的合成。

```

@@ -71,6 +75,7 @@ struct lt6911uxc {
    struct v4l2_dv_timings timings;
    struct v4l2_fwnode_bus_mipi_csi2 bus;
    struct v4l2_subdev sd;
+   struct rkmodule_multi_dev_info multi_dev_info;
    const char *len_name;
    const char *module_facing;
    const char *module_name;

```

```

@@ -763,7 +789,9 @@ static int lt6911luxc_g_mbus_config(struct v4l2_subdev *sd,
unsigned int pad,
    case 4:
        cfg->flags |= V4L2_MBUS_CSI2_4_LANE;
        break;
-
+    case 8:
+        cfg->flags |= V4L2_MBUS_CSI2_4_LANE;
+        break;
    default:
    },
};

@@ -950,7 +975,9 @@ static void lt6911luxc_get_module_inf(struct lt6911luxc
*lt6911luxc,
static long lt6911luxc_ioctl(struct v4l2_subdev *sd, unsigned int cmd, void *arg)
{
    struct lt6911luxc *lt6911luxc = to_state(sd);
+    struct device *dev = &lt6911luxc->i2c_client->dev;
    long ret = 0;
+    struct rkmodule_capture_info *capture_info;

    switch (cmd) {
    case RKMODULE_GET_MODULE_INFO:
@@ -959,6 +986,16 @@ static long lt6911luxc_ioctl(struct v4l2_subdev *sd, unsigned
int cmd, void *arg)
    case RKMODULE_GET_HDMI_MODE:
        *(int *)arg = RKMODULE_HDMIIN_MODE;
        break;
+    case RKMODULE_GET_CAPTURE_MODE:
+        capture_info = (struct rkmodule_capture_info *)arg;
+        if (lt6911luxc->csi_lanes_in_use == 8) {
+            dev_info(dev, "8 lanes in use, set dual mipi mode\n");
+            capture_info->mode = RKMODULE_MULTI_DEV_COMBINE_ONE;
+            capture_info->multi_dev = lt6911luxc->multi_dev_info;
+        } else {
+            capture_info->mode = 0;
+        }
+        break;
    default:
        ret = -ENOIOCTLCMD;
        break;
@@ -975,6 +1012,7 @@ static long lt6911luxc_compat_ioctl32(struct v4l2_subdev *sd,
struct rkmodule_inf *inf;
    long ret;
    int *seq;
+    struct rkmodule_capture_info *capture_info;

    switch (cmd) {
    case RKMODULE_GET_MODULE_INFO:
@@ -1007,6 +1045,21 @@ static long lt6911luxc_compat_ioctl32(struct v4l2_subdev
*sd,
    }
    kfree(seq);
    break;
+    case RKMODULE_GET_CSI_DPHY_PARAM:
+        capture_info = kzalloc(sizeof(*capture_info), GFP_KERNEL);

```

```

+         if (!capture_info) {
+             ret = -ENOMEM;
+             return ret;
+         }
+
+         ret = lt6911luxc_ioctl(sd, cmd, capture_info);
+         if (!ret) {
+             ret = copy_to_user(up, capture_info,
sizeof(*capture_info));
+             if (ret)
+                 ret = -EFAULT;
+         }
+         kfree(capture_info);
+         break;
+
default:
+         ret = -ENOIOCTLCMD;
+         break;
+
@@ -1274,6 +1333,36 @@ static inline int lt6911luxc_parse_of(struct lt6911luxc
*lt6911luxc)
+    }
+    #endif
+
+static int lt6911luxc_get_multi_dev_info(struct lt6911luxc *lt6911luxc)
+{
+    struct device *dev = &lt6911luxc->i2c_client->dev;
+    struct device_node *node = dev->of_node;
+    struct device_node *multi_info_np;
+
+    multi_info_np = of_get_child_by_name(node, "multi-dev-info");
+    if (!multi_info_np) {
+        dev_info(dev, "failed to get multi dev info\n");
+        return -EINVAL;
+    }
+
+    of_property_read_u32(multi_info_np, "dev-idx-l",
&lt6911luxc->multi_dev_info.dev_idx[0]);
+    of_property_read_u32(multi_info_np, "dev-idx-r",
&lt6911luxc->multi_dev_info.dev_idx[1]);
+    of_property_read_u32(multi_info_np, "combine-idx",
&lt6911luxc->multi_dev_info.combine_idx[0]);
+    of_property_read_u32(multi_info_np, "pixel-offset",
&lt6911luxc->multi_dev_info.pixel_offset);
+    of_property_read_u32(multi_info_np, "dev-num",
&lt6911luxc->multi_dev_info.dev_num);
+    dev_info(dev,
+        "multi dev left: mipi%d, multi dev right: mipi%d, combile mipi%d,
dev num: %d\n",
+        lt6911luxc->multi_dev_info.dev_idx[0], lt6911luxc-
>multi_dev_info.dev_idx[1],
+        lt6911luxc->multi_dev_info.combine_idx[0], lt6911luxc-
>multi_dev_info.dev_num);
+
+    return 0;
+}
+
static int lt6911luxc_probe(struct i2c_client *client,
const struct i2c_device_id *id)

```

```

{
@@ -1303,6 +1392,10 @@ static int lt6911luxc_probe(struct i2c_client *client,
    return err;
}

+     err = lt6911luxc_get_multi_dev_info(lt6911luxc);
+     if (err)
+         v4l2_info(sd, "get multi dev info failed, not use dual mipi
mode\n");
+
    err = lt6911luxc_check_chip_id(lt6911luxc);
    if (err < 0)
        return err;

```

2.3 DTS配置

dts参考如下配置，旧版变化差异较大，主要有以下几点：

- 在pipeline上的配置，与单MIPI类似，pipeline的配置应配置成单MIPI场景能接收数据对应的MIPI口，例如4K60双MIPI场景是MIPI2+MIPI4进行拼接，4K30及以下是走单MIPI，由MIPI2完成传输，那么可参考这个dts配置，pipeline使用MIPI2的配置，在转接芯片节点添加multi-dev-info信息说明双MIPI的配置。
- 该版本需要将csi2_dphy0_hw~csi2_dphy1_hw，mipi_dcphy0~mipi_dcphy1，mipi0_csi2_hw~mipi5_csi2_hw等节点都okay起来。
- 该版本csi2_dphy节点不再区分phy的版本，与硬件接口不存在捆绑关系，默认根据mipi0_csi2~mipi5_csi2以及rkcif_mipi_lvds~rkcif_mipi_lvds5绑定硬件接口，对应dts和硬件接口关系如下，csi2_dphy的节点可以根据需要按0~5的顺序取用，与硬件接法没有捆绑。

```

sensor0 -> C/D PHY0(可选择csi2_dphy0~5) -> mipi0_csi2 -> rkcif_mipi_lvds
sensor1 -> C/D PHY1(可选择csi2_dphy0~5) -> mipi1_csi2 -> rkcif_mipi_lvds1
sensor2 -> CSI RX0 clk0 + 2lane[0, 1] or 4lane(可选择csi2_dphy0~5) -> mipi2_csi2 -> rkcif_mipi_lvds2
sensor3 -> CSI RX0 clk1 + 2lane[2, 3] (可选择csi2_dphy0~5) -> mipi3_csi2 -> rkcif_mipi_lvds3
sensor4 -> CSI RX1 clk0 + 2lane[0, 1] or 4lane(可选择csi2_dphy0~5) -> mipi4_csi2 -> rkcif_mipi_lvds4
sensor5 -> CSI RX1 clk1 + 2lane[2, 3] (可选择csi2_dphy0~5) -> mipi5_csi2 -> rkcif_mipi_lvds5

```

使用两路CSI RX0 和CSI RX1拼接的dts配置参考如下：

```

&csi2_dphy0_hw {
    status = "okay";
};

&csi2_dphy1_hw {
    status = "okay";
};

&csi2_dphy0 {
    status = "okay";

    ports {
        #address-cells = <1>;
        #size-cells = <0>;
        port@0 {

```



```

        reg = <0>;
        #address-cells = <1>;
        #size-cells = <0>;

        hdmi_mipi_in: endpoint@1 {
            reg = <1>;
            remote-endpoint = <&lt691luxc_out0>;
            data-lanes = <1 2 3 4>;
        };
    };

port@1 {
    reg = <1>;
    #address-cells = <1>;
    #size-cells = <0>;

    csiphy0_out: endpoint@0 {
        reg = <0>;
        remote-endpoint = <&mipi2_csi2_input>;
    };
};

};

&i2c3 {
    status = "okay";

    lt691luxc: lt691luxc@2b {
        compatible = "lontium,lt691luxc";
        status = "okay";
        reg = <0x2b>;
        clocks = <&ext_cam_clk>;
        clock-names = "xvclk";
        power-domains = <&power RK3588_PD_VI>;
        pinctrl-names = "default";
        pinctrl-0 = <&lt691luxc_pin>;
        interrupt-parent = <&gpio1>;
        interrupts = <RK_PB3 IRQ_TYPE_LEVEL_LOW>;
        // reset-gpios = <&gpio1 RK_PB1 GPIO_ACTIVE_LOW>;
        // power-gpios = <&gpio1 RK_PA6 GPIO_ACTIVE_HIGH>;
        plugin-det-gpios = <&gpio1 RK_PB4 GPIO_ACTIVE_HIGH>;
        rockchip,camera-module-index = <0>;
        rockchip,camera-module-facing = "back";
        rockchip,camera-module-name = "HDMI-MIPI2";
        rockchip,camera-module-lens-name = "LT691UXC";

        multi-dev-info {
            dev-idx-l = <4>;
            dev-idx-r = <2>;
            combine-idx = <2>;
            pixel-offset = <0>;
            dev-num = <2>;
        };

        port {
            lt691luxc_out0: endpoint {
                remote-endpoint = <&hdmi_mipi_in>;
                data-lanes = <1 2 3 4>;
            };
        };
    };
};

```

```

    };

};

&mipi_dcphy0 {
    status = "okay";
};

&mipi_dcphy1 {
    status = "okay";
};

&mipi0_csi2_hw {
    status = "okay";
};

&mipi1_csi2_hw {
    status = "okay";
};

&mipi2_csi2_hw {
    status = "okay";
};

&mipi3_csi2_hw {
    status = "okay";
};

&mipi4_csi2_hw {
    status = "okay";
};

&mipi5_csi2_hw {
    status = "okay";
};

&mipi2_csi2 {
    status = "okay";

    ports {
        #address-cells = <1>;
        #size-cells = <0>;

        port@0 {
            reg = <0>;
            #address-cells = <1>;
            #size-cells = <0>;

            mipi2_csi2_input: endpoint@1 {
                reg = <1>;
                remote-endpoint = <&csidphy0_out>;
            };
        };

        port@1 {
            reg = <1>;
            #address-cells = <1>;
            #size-cells = <0>;

```

```

        mipi2_csi2_output: endpoint@0 {
            reg = <0>;
            remote-endpoint = <&cif_mipi_in2>;
        };
    };
};

&rkCIF_mipi_lvds2 {
    status = "okay";

    port {
        cif_mipi_in2: endpoint {
            remote-endpoint = <&mipi2_csi2_output>;
        };
    };
};

&rkCIF {
    status = "okay";
};

&rkCIF_mmu {
    status = "okay";
};

```

2.4 转接芯片配置说明

转接芯片部分的配置多了multi-dev-info部分配置，该部分配置与驱动的rkmodule_multi_dev_info结构体对应；

multi-dev-info配置解释如下：

dev-idx-l: 左半边图像对应的mipi接口编号

dev-idx-r: 右半边图像对应的mipi接口编号

combine-idx: 将左右两张图想合并到其中的一个idx，dts的链路以这个idx为准

pixel-offset: 像素偏移，默认设置0

dev-num: 多少个mipi设备进行拼接

对应驱动获取dts配置的配置：

```

static int lt6911uxc_get_multi_dev_info(struct lt6911uxc *lt6911uxc)
{
    struct device *dev = &lt6911uxc->i2c_client->dev;
    struct device_node *node = dev->of_node;
    struct device_node *multi_info_np;

    multi_info_np = of_get_child_by_name(node, "multi-dev-info");
    if (!multi_info_np) {
        dev_info(dev, "failed to get multi dev info\n");
        return -EINVAL;
    }
}

```

```

of_property_read_u32(multi_info_np, "dev-idx-l",
    &lt691luxc->multi_dev_info.dev_idx[0]);
of_property_read_u32(multi_info_np, "dev-idx-r",
    &lt691luxc->multi_dev_info.dev_idx[1]);
of_property_read_u32(multi_info_np, "combine-idx",
    &lt691luxc->multi_dev_info.combine_idx[0]);
of_property_read_u32(multi_info_np, "pixel-offset",
    &lt691luxc->multi_dev_info.pixel_offset);
of_property_read_u32(multi_info_np, "dev-num",
    &lt691luxc->multi_dev_info.dev_num);
dev_info(dev,
    "multi dev left: mipi%d, multi dev right: mipi%d, combile mipi%d, dev
num: %d\n",
    lt691luxc->multi_dev_info.dev_idx[0], lt691luxc-
>multi_dev_info.dev_idx[1],
    lt691luxc->multi_dev_info.combine_idx[0], lt691luxc-
>multi_dev_info.dev_num);

return 0;
}

```

参考补丁包的: rk3588-evb1-lp4-lt691luxc-dual-mipi.dts

注: 补丁包提供的dts配置是dphy0和dphy1两路进行拼接, 对应的mipi口分别为mipi2和mipi4

3. 调试命令

3.1 查看拓扑

使用media-ctl 工具可以查看pipeline的结构是否正常, pipeline错误, 大概率是dts配置错误。

```
media-ctl -d /dev/mediaX -p //X=0123...
```

3.2 抓图命令

开启数据流命令参考:

```
v4l2-ctl --verbose -d /dev/video0 --set-fmt-
video=width=3840,height=2160,pixelformat='NV12' --stream-mmap=4
```

抓图命令参考:

```
v4l2-ctl --verbose -d /dev/video0 --set-fmt-
video=width=3840,height=2160,pixelformat='NV12' --stream-mmap=3 --stream-skip=4 -
--stream-to=/data/3840x2160p60_nv12-1.yuv --stream-count=5 --stream-poll
```

4. 常见问题

4.1 dts配置是否需要配置两个链路？

硬件上使用两路MIPI数据流，但是dts不需要配置两个链路，仅需要在驱动中确认是否打开双MIPI的模式，以及增加rkmodule_multi_dev_info结构体参数的配置，平台的框架代码会根据该信息对两路数据流进行拼接，dts只需要配置单MIPI的场景下可以正常出数据流的链路即可。

4.2 应用层是否需要重新适配？

dts是配置一条链路，因此对用户空间而言，pipeline是一致的，具体的可以通过如下命令查看pipeline：

```
media-ctl -d /dev/mediaX -p //X=0123...
```

4.3 抓图异常

不出图分为两类问题，抓不到图串口没有报错和抓不到图串口出现报错

4.3.1 抓不到图且串口没有报错

执行抓图命令，出现select timeout，说明没有数据，可以从以下点进行排查：

- dts接口是否配置错误，尤其是拼接的两路MIPI以及对应的链路配置
- 转接芯片是否没有MIPI信号输出，需要确认两路MIPI都有信号输出
- MIPI信号是连续模式还是非连续模式，如是连续模式，TX端的数据流开早了

4.3.2 抓不到数据MIPI相关报错

抓不到数据，比较常见的是出现MIPI相关的报错，CRC ECC等错误，可以排查以下问题：

- 驱动代码 link freq的配置，该配置为mipi clk，即速率的一半，需要与TX端的MIPI 速率对应配置
- dts MIPI lane数是否配置错误
- TX端是否只有一个MIPI PORT有MIPI信号输出
- MIPI信号质量问题
- TX端的MIPI信号问题